STAGE 2 – PRACTICAL ENGINEERING CHALLENGE

TEAM ID:	TEAM NAME:
LRC-26-0144	FABULOUS

Problem Statement: Industrial Manufacturing Process Automations

Industrial manufacturing depends on highly controlled liquid-handling systems to deliver process fluids reliably from a source or bulk supply to downstream process equipment. Depending on the application, these fluids may include **ultrapure fluids, process chemicals, solvents, developers, or CMP-related fluids.** Industry standards explicitly address liquid-chemical distribution, high-purity piping, bulk chemical distribution, system qualification, and UPW system operation and monitoring.

In such systems, **flow is a fundamental process variable**. An incorrect flow rate can indicate or cause a restriction, incorrect valve state, inadequate pumping performance, abnormal process demand, or another hydraulic deviation. The consequence may be unstable process conditions, incorrect chemical delivery, equipment malfunction, or loss of process performance.

This challenge therefore focuses on a fundamental industrial facilities and process-engineering task: **To establish a defined process-fluid flow, measure it reliably, detect a deviation, diagnose its cause, and restore the process to the required operating condition.**

Engineering Challenge

As you are part of a multidisciplinary engineering team responsible for designing, building and validating a small-scale automated liquid-flow system represented of industrial process handling system.

CASE 1 – CONTROL + MEASUREMNT

Problem: Develop a system that manages the changing level of fluid in the sump automatically (when there is a sudden external fluid charging to the sump) and while accurately measuring and totalizing the fluid discharged from the sump using the flow transmitter.

CASE 2 – FLUSHING +SHUTDOWN

Problem: Develop an alternative operating configuration that allows flush water to pass through the system while all three pumps operate simultaneously, and then safely transition the system to an OFF condition when the flush-water supply has been exhausted.

Given Process Parameters

Pipe diameter =	**8mm**
Fluid density (ρ) =	**1000kg/m^3**
Dynamic Viscosity (μ) =	**1x10^{-3}Pa.s**
Required Flow range =	**1-6 L/min**
Nominal target flow for the live challenge =	**3L/min**

Available Hardware

Mechanical Hardware	Electronic Hardware
1. Water Tank – 3 Quantity	1. Custom PCB
2. Pumps – 3 Quantity	2. ESP32 NodeMCU module
3. FT-01 Flow Transmitter – 01 Quantity	3. Pump Driver-1
4. Ball valve – 3 Quantity	4. Pump Driver-2
5. ON/OFF/XV Valve – 1 Quantity	5. Pump Driver-3
6. 10-mm OD tubing/blue – 9 meters/1 meter	6. 12 V DC power input (**SMPS**)
7. Microcontroller/PCB/ data-acquisition software	7. 12 VDC Solenoid-valve interface
8. Required electrical/ electronic accessories	8. Flow-sensor interface
	9. Float-switch interface
	10. ON/OFF-switch interface
	11. Pump-driver PWM/enable interfaces
	12. On/Off Switch

The team must determine the appropriate arrangement, connections and measurement method.

- Assemble and commission the liquid-flow system using the components provided.
- Measure and characterize the flow using FT-01
- Interface FT-01 with the provided **data-acquisition system**
- Experimentally determine the relationship between sensor output and actual flow.
- Characterize the effect of valve position on flow rate. Record and analyse flow vs time data
- Maintain target flow of 3L/min
- Use graphs and measured data to justify engineering decisions
- Design an appropriate hydraulic arrangement and P&ID

TEAM INSTRUCTIONS

- As a team go through the list provided to read about each component that has been Listed.
- The PCB is a 12 V DC and the SMPS connections should be made carefully.

TASK 1: HYDRAULICS SYSTEM DESIGN

Q1.1 Design and build the hydraulic path with the flow transmitter on the recirculation line. Submit/sketch a labelled P&ID below by showing the pumps, Needle valves, flow transmitter, flow direction, Tank, sump, tube diameter and measurement points.

CASE 1 – Process Fluid Transfer & Automatic Level Protection

Objective: Set-up and operate a closed-loop process- fluid handling system using the available tank, pump, valves flow transmitter, sump, float sensor and control PCB. The System must continuously transfer process fluid recirculation system by connecting the 10L fluid tank with a On/Off valve to a pump-01 and the pump-01 goes to the inlet of sump 4.2L tank. The Sump 4.2L outlet line is connected to another pump-02 followed by a flow transmitter that measures the total flow back to the 10L fluid tank. The Pump-01 runs on 50% duty cycle and Pump-02 runs on 100% duty cycle. **Calibrate the flow transmitter** and demonstrate correct fluid circulation, if there is sudden introduction of external fluid into the Sump tank then Pump-01 should stop while pump – 02 should still runs till fluid level reduces in the sump. After the fluid reduction in the SUMP 4.2L tank the Pump-01 should turn on and the process must continue. **Also, demonstrate Flow vs Time trends in mL/s, total transferred Liters back to the 10L tank** (share Screenshots flow in L/min). **Share the code**. The case-1 can be turned on using the available on/off Switch, to start the system.

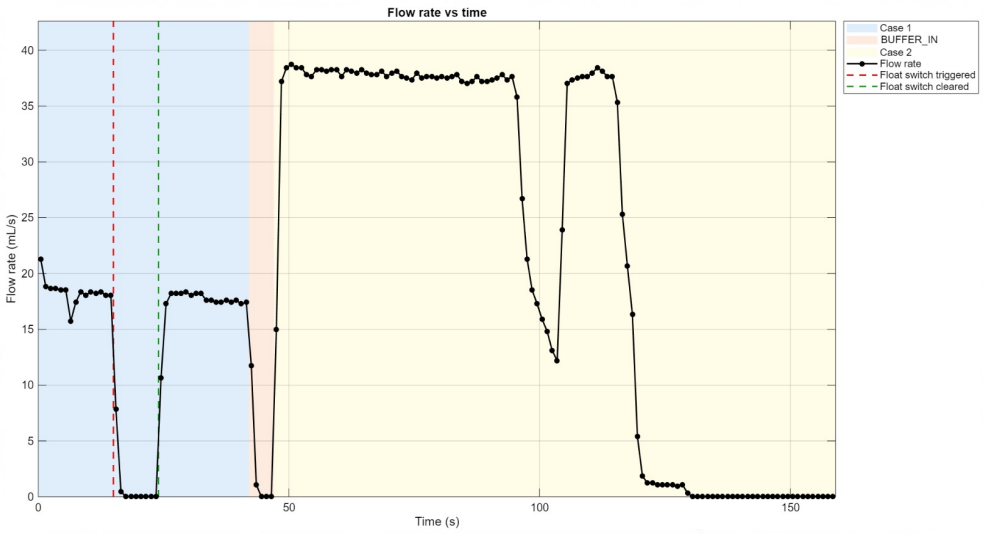
CASE 2 – Flush/Line Conditioning Sequence

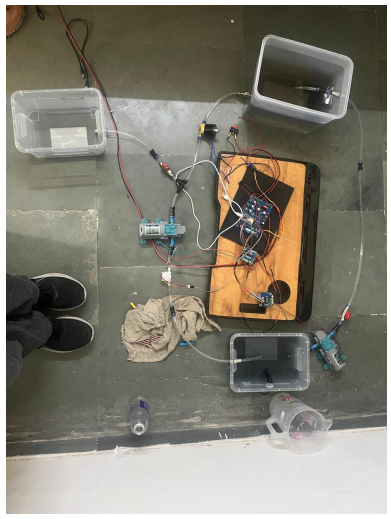
Objective: After successfully completing case 1, initiate the flush water in the sequence using the **equal Tee.** The On/Off valve must be switched off in order to isolate the 10L process fluid tank. The flush Fluid tank (4.2L) is used as a temporary fluid source to flush out the process fluid or any debris from the system. Before the start of this process make sure you open the manual ball valve on the fluid flush line before Pump-03 starts. The entire process of case -1 and case - 2 should run in a single go without manual intervention except opening of ball valve before the Pump-03 starts. **The System must stop as soon as the tank flush fluid 4.2L tank finishes.** The system must be automated without human interventions (only ball valve can be opened manually before the start of flush line sequence)

SUBMIT:

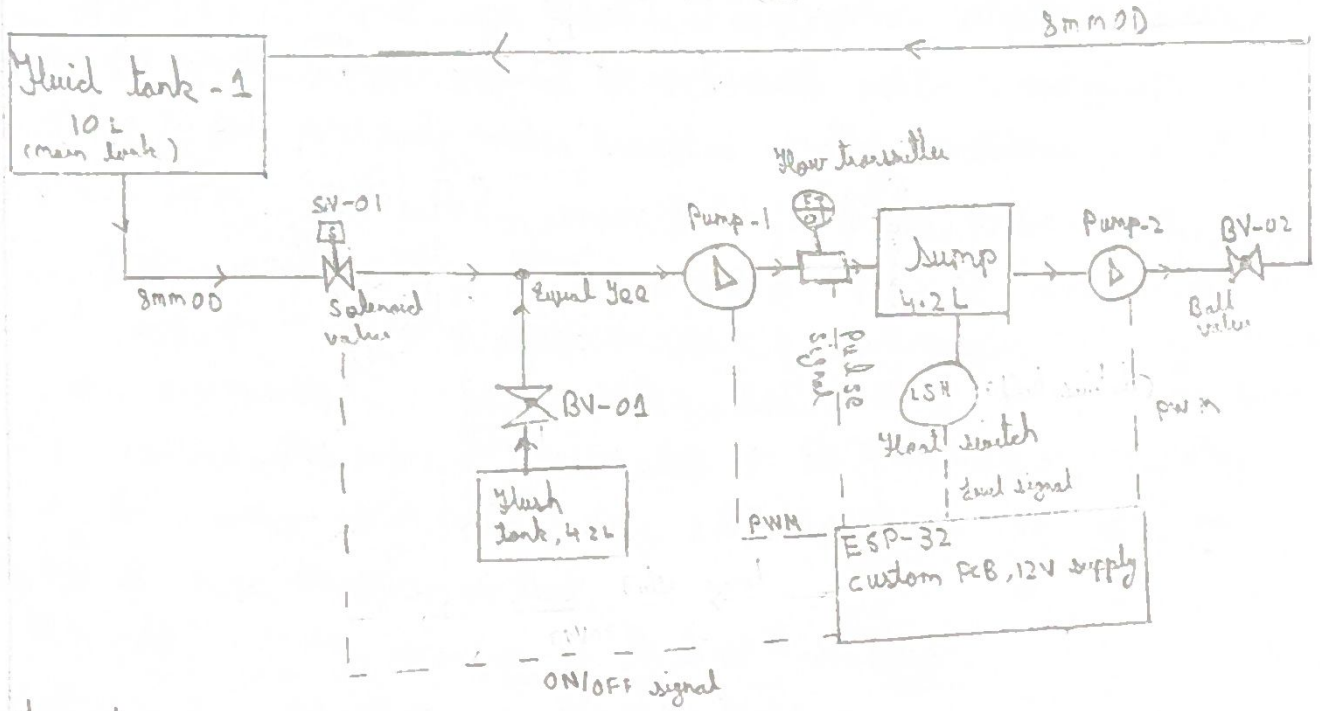
A. Submit: 10-15 min Video and Photograph of the completed setup, with identification of each component and explain the direction of flow in the video verbally. Demonstrate both Case - 1 and Case – 2.

B. Final P&ID Block diagram





R) P and ID :



Legend

—— : process line, 8mm OD clear tubing

- - - : electrical signal to or from ESP 32

> : flow direction

: solenoid valve

: manual ball valve

Process :

Case 1 : i) BV-01 closed , Pump 1 at 50% duty cycle , Pump 2 at 100% duty cycle
SV-01 open

ii) float switch trigger => Pump 1 off until switch no longer triggered

case 2 : pump 1 runs at 100% duty cycle , pump 2 runs at 50% duty cycle.
pump 1 stops if LSH (float switch) triggers or FT-01 (flow sensor) reads less than 0.1 L/min for 7 seconds.

A Total transferred volume back to ~~case~~ main tank during case 1 : $\frac{0.5\,L}{10.61} \times 17.4$

$= 0.81\,L$

Q1.2 After setting-up the system, now operate it at approximately at 1L/min, 3L/min & 6L/min. Compare the theoretical velocity with the actual operating behaviour.

The Pumps theoretical maximum velocity is supposed to be 6L/min, which should be achieved when running at a 100% duty cycle. However under practical testing conditions, at the 100% duty cycle while connected to the pipes, it was only achieving 2·5L/min. Removing all pipes got it up to 4·5 L/min, & so we can attribute the drop in velocity to the resistances of all the flow components i.e. Pipes, couplers, ball valve etc. With this in mind, the actual flowrates at 50% and 16·67% duty cycle (corresponding to 3L/min, 1 L/min ideal flowrate) were also lower than expected.

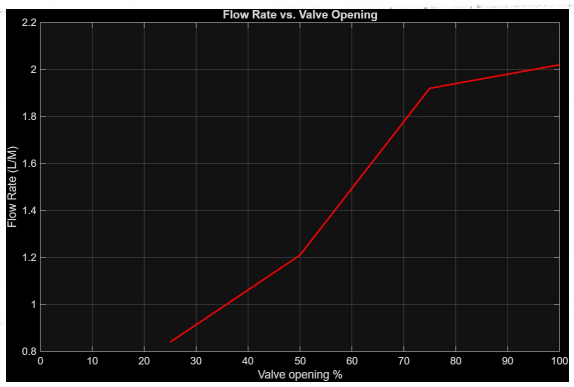
Duty Cycle	Expected flowrate	Realised Flowrate.
100%	6L/min	2·5 L/min
50%	3L/min	1·21 L/min
16·67%	1L/min	0.3 L/min.

This is consistent with our expectations of how the resistance has decreases the flowrate.

FLOW RESPONSE TO VALVE POSITION

Q 2.1 Set the ball valve to approximately 25%,50%,75% & 100%. At each position, allow the flow to stabilize and record the flow rate. Create the following graph of valve position (%) vs Flow Rate (L/min)

VALVE POSITION	FLOW RATE
25%	0.84 L/m
50%	1.21 L/m
~~70%~~ 75%	1.92 L/m
100%	2.02 L/m



Q2.2 Determine the experimental calibration equation: $Q = af + b$, where Q = flow rate in L/min, f = pulse frequency in Hz, a = experimental calibration coefficient and b = offset.

Report:

R^2 =	0.99705
a =	9.582×10^{-3} L/min Hz
b =	-5.383×10^{-2} L/min
Maximum calibration error =	7.885% (absolute error = 0.1099 L/min)
Average calibration error =	3.1199% (absolute average error = 0.029 L/min)

Q2.3 FT-01 produces pluses corresponding to water flow. Run the system at least for 5 different flow conditions within the available operating range (Hint: use formulas f=N/t and Q=V/t). For each condition, record the following:

Trial	Pulses	Time(s)	Frequency (Hz)	Collected Volume (L)	Actual Flow (L/min)
1	3433	12.865	266.848	0.5	2.536
2	32854	14.34	218.419	0.5	2.092
3	3368	20.895	161.264	0.5	1.486
4	3312	31.389	105.515	0.5	0.972
5	3530	111.903	31.545	0.5	0.269

Q2.2) OBSERVATION

Q (L/min)	2.536	2.536	2.488	2.092	2.033	1.486	1.394	1.217
f (Hz)	2.487	266.948	265.409	218.419	218.298	161.264	162.578	134.916

Q (L/min)	0.972	0.3	0.269
f (Hz)	105.514	34.666	31.345

→ if $Q = A \cdot f + B$: i) $A = \dfrac{n\sum x_i y_i - \sum x_i \sum y_i}{n\sum x_i^2 - (\sum x_i)^2} = 9.582 \times 10^{-3}$ L/min·Hz

(or $A^{-1} = 104.363$ min·Hz/L)

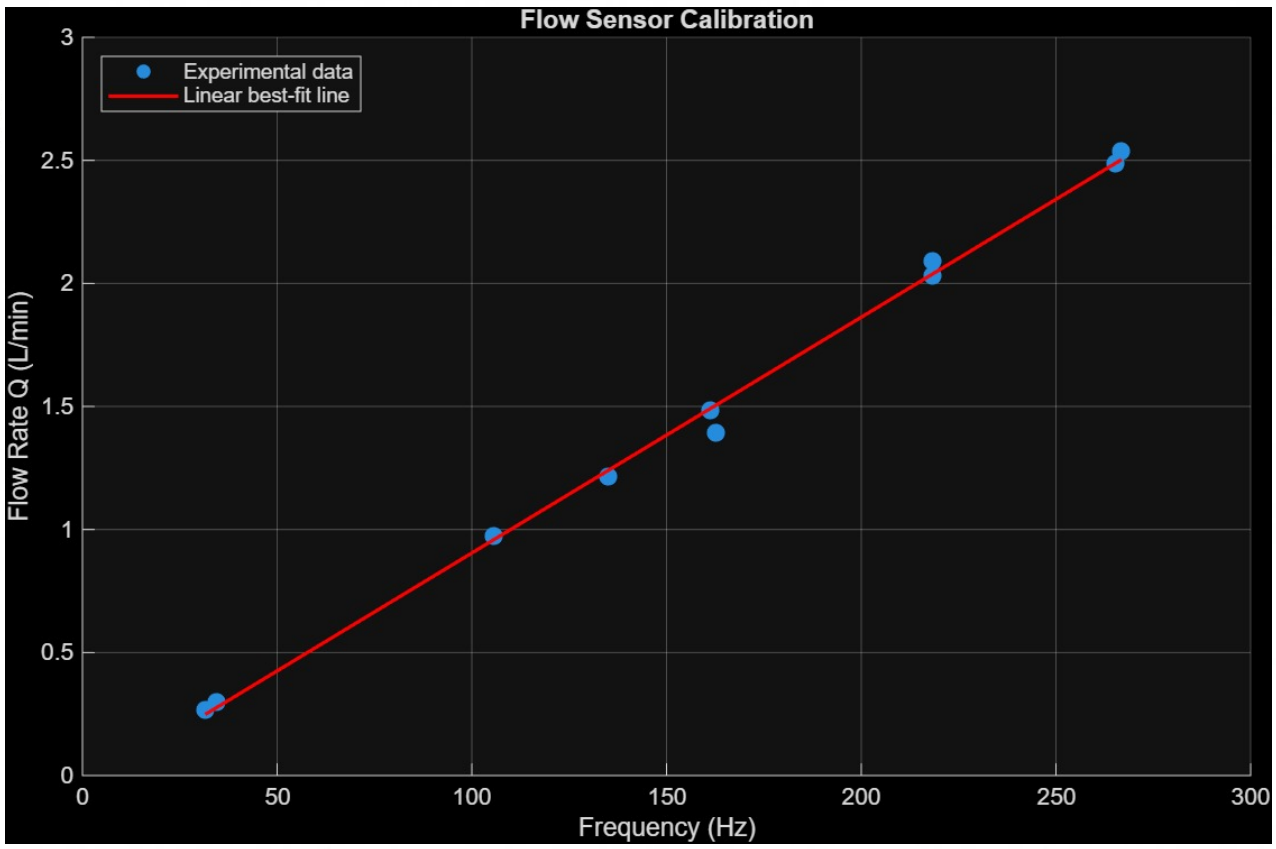
ii) $B = \bar{Q} - A\bar{f} = -5.383 \times 10^{-2}$ L/min

iii) $R^2 = 1 - \left[\sum (Q_i - \hat{Q}_i)^2 / \sum (Q_i - \bar{Q}_i)^2\right] = 0.99705$

where $\hat{Q}_i$ is predicted Q

iv) max error = ~~0.10992~~ 0.10992 L/min

avg error = 0.02897 L/min



TASK 3: HYDRAULIC SYSTEM THEORY AND FAULT ANALYSIS

Q3.1 The effective liquid volume inside a section of tubing is 0.5L, while the operating flow rate is 2.5L/min. The hydraulic residence time is given by $T_r = Q/V$. Calculate the residence time in seconds, also answer the following questions:

1. What does residence time physically represent in this system?
2. What happens to residence time if the tubing volume is doubled while the flow rate remains constant?
3. What happens to residence time if the flow rate is doubled while the tubing volume remains constant?
4. Why is residence time important when interpreting a sudden change in measured flow?

Eff. Vol. = V (L)

operating rate = Q (L/min)

$\Rightarrow$ Residence Time = $\frac{Q}{V} = \frac{V}{\cancel{L/min}} \frac{L/min}{L} = min^{-1}$

$\therefore$, to calculate T_r in seconds, we have to use inverse of above eq. :

$$T_r = \frac{V}{Q} = \frac{0.5}{2.5} min = \frac{1}{5} min = \frac{60}{5} s = \underline{12 secs}$$

(a) Residence time is the avg. time a unit volume of liquid spends inside a given section of tubing, from entering to leaving.

(b) $T_r \propto V \Rightarrow$ if $V' \longrightarrow 2V$ then $T_r' = 2 \cdot T_r = \underline{24 secs}$
$\therefore$, T_r doubles too

(c) $T_r \propto 1/Q \Rightarrow$ if $Q' = 2Q$ then $T_r' = T_r/2 = 6$ seconds
$\therefore$, T_r halves

(d) T_r is the time it takes for any sudden change at i/p of pipe to be detected at pipe's o/p. Therefore, if any sudden change is detected at o/p, its cause may have entered T_r seconds ago & may no longer be visible at i/p.

Q3.2 The following observation are obtained from the same system:

CONDITION	PUMP	VALVE	FT-01 PULSE SIGNAL	MEASURED FLOW
A	ON	Open	Normal	Normal
B	ON	Partially closed	Reduced	Reduced
C	ON	Open	Zero	Zero
D	OFF	Open	Zero	Zero
E	ON	Open	Intermittent	Fluctuating

For each of the condition, identify the most likely physical or instrumentation condition. Then answer: Which condition is most difficult to distinguish using FT-01 alone, and what additional information would you require to distinguish the fault? (Hint: Teams must consider possibilities such as: Valve restriction, pump failure, hydraulic blockage, sensor/signal failure, unstable flow etc)

ANSWER: Assumption: Measured Flow calculated using FT-01 signal, not physically

A —> Normal, expected operation, both sensor & pump operate correctly

B —> Partially closed valve adds resistance to flow, causing flow to fall & pump throttle; expected op.

C —> Either no flow or sensor / measurement error :
(i) Pump Failure : motor error, driver, ESP32 or SMSP
(ii) Sensor —"— : rotor jammed, disconnected / faulty i/p wiring

D —> Nothing drives liq., hence no flow is expected; Expected operation.

E —> This is improper operation & can occur due to :
(i) Air entering system : tank is empty, unprimed pump, suction leak, pump surge
(ii) Measurement error : loose connecting pipes, intermittent wiring or noise corrupting the signal

Condition C appears to be most difficult to distinguish using only FT-01 since zero pulse carry absolutely no information about whether flow has truly stopped or if sensor fails to report flow.
While ~~more~~ failures are more varied in E than in C, since fluctuating pulses still hold some information. For eg., impossible rates of pulse imply electrical noise, irregular dips within normal rates imply air in system or unstable pumping. E can also be tested by switching pump off: if pulse continue, electrical issue, if pulse stops then hydraulic causes. Same Test in C reveals nothing.
Additional information C can use :
(i) Tank & sump liq. levels & trends : Tells actual flow of system
(ii) ~~Pump Discharge Pressure~~ : ~~Provides i/p —> o/p pressure of pump to isolated whether~~ error in pump, tubing or sensor
(iii) Pump Motor Current : if issue in pump, tells if issue is in pump, or driver or ~~ESP32~~.

HARDWARE LIST

Mechanical Hardware

Component	Description	Main Use
Ball Valve (BV-01)	A manually operated valve with a spherical shut-off mechanism.	Used to **manually isolate or permit fluid flow** in the pipeline. It is useful for maintenance, testing, and flow isolation.
10-Litre Fluid Tank (Fluid Tank-01)	Transparent storage tank with approximately 10 L capacity.	Used as the **main fluid reservoir** for storing process fluid. The transparent body also allows the fluid level and condition to be visually monitored.
5-Litre Sump	Small transparent reservoir labelled **SUMP**, approximately 4.5 L.	Used as a **collection/storage vessel** for fluid returning to or entering the sump section of the system.
4.2-Litre Flush Fluid Tank / Flush Line	The transparent vessel labelled **FLUSH FLUID** is approximately 4.5 L.	Provides **flush fluid** for cleaning or flushing the main pipe circuit when required.
ON/OFF Valve (OV-01)	Electrically operated valve/solenoid-type valve shown in the setup.	Used for **automatic control of fluid flow**, allowing the controller to turn a particular flow path on or off.
Pumps -Pump-1, Pump-2, Pump-3	Motor-driven fluid pumps. Three pumps are shown in the setup.	Used to **circulate and transfer fluid** between different sections of the system. Each pump can be independently controlled.
8 mm Process Pipe	Flexible blue tubing with approximately 8 mm process-line size.	Used as the **main fluid-flow path** connecting tanks, valves, pumps, tees, and other components.
10 mm × 8 mm Reducers	Reducer fittings that transition between a 10 mm connection and an 8 mm pipe.	Used to **adapt the pump connection size to the 8 mm process pipe**. **Mainly used for Pumps**
8 mm Connectors	Push-fit/barbed-type fittings compatible with the 8 mm tubing.	Used to **join pipes to components** and to make detachable connections between sections of tubing.
8 mm Equal Tee	T-shaped connector having three equal 8 mm ports.	Used to **split one 8 mm fluid line into two branches or combine two branches into one line.**

Electronic Hardware

Electronic Component	Identification / Function	Use
PCB -Printed Circuit Board	Custom control board	Provides the electrical interconnection between the 12 V power supply, ESP32 controller, **sensors, valves, indicators, and pump-driver interfaces.**
ESP32 / NodeMCU ESP32S module (U7)	Main microcontroller	Acts as the **central controller**. It provides GPIO/PWM signals for controlling outputs and receiving signals from sensors/switches.

Electronic Component	Identification / Function	Use
12 V DC Input Connector	Marked **12V INPUT**	Connection point for the external 12 V DC power supply.
12 V Output Connectors	P1, P2, P3 and additional 12 V output terminals	Provide 12 V power connections for external loads such as the pump-driver/motor circuits.
Motor Driver Interfaces -U2, U3, U4	Three multi-pin driver connectors shown on the PCB	Provide dedicated interfaces for **Pump-1, Pump-2 and Pump-3 driver/control connections.**
Solenoid Valve Connector	Labelled **SOLENOID_VALVE -S/V/G**	Interface for controlling/connecting the solenoid or ON/OFF valve.
Flow Sensor Connector	Labelled **FLOW_SENSOR - S/V/G**	Interface for the water-flow sensor; supplies power and receives the sensor signal. **THE FLOW SENSOR SHOULD BE CALIBRATED**
Float Switch Connector	Labelled **FLOAT_SWITCH -FSW/GND**	Interface for a float/level switch used to detect a water-level condition.
ON/OFF Switch Connector	Labelled **ON/OFF_SWITCH -SW/GND**	Interface for an external manual ON/OFF switch.
12V DC SMPS	**12V DC SMPS**	For Power Supply to the board

```
#include <Arduino.h>

const uint8_t P1_REN = 22, P1_LEN = 21, P1_RPWM = 25, P1_LPWM = 23;
const uint8_t P2_REN = 26, P2_LEN = 13, P2_RPWM = 33, P2_LPWM = 32;
const uint8_t VALVE = 27, FLOAT_PIN = 35, FLOW = 14;
const uint8_t BUTTON = 34;
const int C1_P1_PWM = 127;
const int C1_P2_PWM = 255;
const int C2_P1_PWM = 255;
const int C2_P2_PWM = 127;
const uint32_t KICK_MS = 300;

const int VALVE_OPEN = 255;
const int VALVE_CLOSED = 0;
const int FLOAT_ACTIVE = LOW;
const int BUTTON_PRESSED = LOW;
const bool FLOAT_STOPS_P2 = false;
const bool FLUSH_FLOAT_STOPS_P1 = true;
const float NOFLOW_LPM = 0.1f;

const uint32_t DEBOUNCE_MS  = 300;
const uint32_t BTN_DEBOUNCE_MS = 50;
const uint32_t BUFFER_MS    = 5000;
const uint32_t NOFLOW_S     = 7;
const float   CAL_K = 108.0f;

enum Mode { STATE1, BUFFER_IN, FLUSH, BUFFER_OUT };
const char *MODE_NAME[] = { "STATE1", "BUFFER_IN", "FLUSH", "BUFFER_OUT" };
Mode mode = STATE1;
uint32_t modeStart = 0;

bool pumpsRequested = false;
bool floatActive = false, floatRaw = false;
uint32_t floatChanged = 0;
bool btnRaw = false, btnState = false; uint32_t btnChanged = 0;
bool valveIsOpen = false;
bool p1NoFlowOff = false;
uint32_t zeroSecs = 0;

struct Pump {
  uint8_t ren, len, rpwm, lpwm;
  int cur;
  bool kicking; uint32_t kickEnd;
};
Pump pump1 = { P1_REN, P1_LEN, P1_RPWM, P1_LPWM, 0, false, 0 };
Pump pump2 = { P2_REN, P2_LEN, P2_RPWM, P2_LPWM, 0, false, 0 };

volatile uint32_t pulses = 0, lastUs = 0;
void IRAM_ATTR onPulse() {
  uint32_t now = micros();
  if (now - lastUs >= 300) { pulses++; lastUs = now; }
}
uint32_t readPulses() { noInterrupts(); uint32_t p = pulses; interrupts(); return p; }

uint32_t runStartMs = 0, runStartP = 0;
uint32_t nextTick = 0, lastP = 0, sec = 0;

void pumpTo(Pump &p, int pwm) {
  if (pwm == p.cur) return;
  if (pwm == 0) {
    analogWrite(p.rpwm, 0);
    digitalWrite(p.ren, LOW); digitalWrite(p.len, LOW);
    p.kicking = false;
  } else {
    digitalWrite(p.lpwm, LOW);
    digitalWrite(p.ren, HIGH); digitalWrite(p.len, HIGH);
```

```
    if (p.cur == 0 && pwm < 255) {
      analogWrite(p.rpwm, 255);
      p.kicking = true; p.kickEnd = millis() + KICK_MS;
    } else if (!p.kicking) {
      analogWrite(p.rpwm, pwm);
    }
  }
  p.cur = pwm;
}

void serviceKick(Pump &p, uint32_t now) {
  if (p.kicking && (int32_t)(now - p.kickEnd) >= 0) {
    p.kicking = false;
    if (p.cur > 0) analogWrite(p.rpwm, p.cur);
  }
}

void update() {
  bool openValve = false, want1 = false, want2 = false;
  int pwm1 = 0, pwm2 = 0;

  switch (mode) {
    case STATE1:
      openValve = !floatActive;
      want1 = pumpsRequested && !floatActive;
      want2 = pumpsRequested && !(FLOAT_STOPS_P2 && floatActive);
      pwm1 = C1_P1_PWM; pwm2 = C1_P2_PWM;
      break;
    case BUFFER_IN:
    case BUFFER_OUT:
      openValve = false; want1 = false; want2 = false;
      break;
    case FLUSH:
      openValve = false;
      want1 = pumpsRequested && !p1NoFlowOff && !(FLUSH_FLOAT_STOPS_P1 && floatActive);
      want2 = pumpsRequested;
      pwm1 = C2_P1_PWM; pwm2 = C2_P2_PWM;
      break;
  }

  analogWrite(VALVE, openValve ? VALVE_OPEN : VALVE_CLOSED);
  valveIsOpen = openValve;

  pumpTo(pump1, want1 ? pwm1 : 0);
  pumpTo(pump2, want2 ? pwm2 : 0);
}

void report() {
  Serial.printf("[%s] float_pin=%d float=%s valve=%s P1=%d%% P2=%d%%\n",
                MODE_NAME[mode], digitalRead(FLOAT_PIN),
                floatActive ? "ACTIVE" : "clear",
                valveIsOpen ? "OPEN" : "CLOSED",
                pump1.cur * 100 / 255, pump2.cur * 100 / 255);
}

void setMode(Mode m) {
  mode = m;
  modeStart = millis();
  if (m == FLUSH) { p1NoFlowOff = false; zeroSecs = 0; }
  Serial.printf(">>> %s\n", MODE_NAME[m]);

  if (m == BUFFER_IN) Serial.println(">>> OPEN the manual flush valve now (5 s)");
  if (m == BUFFER_OUT) Serial.println(">>> CLOSE the manual flush valve now (5 s)");
  update();
  report();
}
```

```
134
135 void startRun() {
136   runStartMs = millis();
137   runStartP = lastP = readPulses();
138   nextTick = runStartMs + 1000;
139   sec = 0;
140 }
141
142 void endRun() {
143   uint32_t total = readPulses() - runStartP;
144   float secs  = (millis() - runStartMs) / 1000.0f;
145   float avgHz = secs > 0 ? total / secs : 0;
146   float avgLpm = avgHz / CAL_K;
147   float volL  = total / (CAL_K * 60.0f);
148
149   Serial.printf("=== RUN END: duration=%.1f s total_pulses=%lu avg_hz=%.2f "
150                 "avg_flow=%.3f L/min total_volume=%.3f L ===\n",
151                 secs, (unsigned long)total, avgHz, avgLpm, volL);
152 }
153
154 bool buttonPressed(uint32_t now) {
155   bool r = digitalRead(BUTTON) == BUTTON_PRESSED;
156   if (r != btnRaw) { btnRaw = r; btnChanged = now; }
157   if (r != btnState && now - btnChanged >= BTN_DEBOUNCE_MS) {
158     btnState = r;
159     return btnState;
160   }
161   return false;
162 }
163
164 void setup() {
165   for (uint8_t p : {P1_REN, P1_LEN, P1_RPWM, P1_LPWM, P2_REN, P2_LEN, P2_RPWM, P2_LPWM}) {
166     pinMode(p, OUTPUT); digitalWrite(p, LOW);
167   }
168   pinMode(FLOAT_PIN, INPUT);
169   pinMode(BUTTON, INPUT);
170   pinMode(FLOW, INPUT_PULLUP);
171   attachInterrupt(digitalPinToInterrupt(FLOW), onPulse, RISING);
172
173   Serial.begin(115200);
174   delay(500);
175   floatRaw = floatActive = digitalRead(FLOAT_PIN) == FLOAT_ACTIVE;
176   btnRaw = btnState = digitalRead(BUTTON) == BUTTON_PRESSED;
177   update();
178   Serial.println("\nType r = start, x = stop. Button steps STATE1 -> FLUSH -> STATE1.");
179   report();
180 }
181
182 void loop() {
183   uint32_t now = millis();
184
185   if (Serial.available()) {
186     char c = Serial.read();
187     if ((c == 'r' || c == 'R') && !pumpsRequested) {
188       pumpsRequested = true; mode = STATE1; update(); report(); startRun();
189     }
190     if ((c == 'x' || c == 'X') && pumpsRequested) {
191       pumpsRequested = false; mode = STATE1; update(); report(); endRun();
192     }
193   }
194
195   if (buttonPressed(now) && pumpsRequested) {
196     if (mode == STATE1)   setMode(BUFFER_IN);
197     else if (mode == FLUSH) setMode(BUFFER_OUT);
198   }
199
200   if (mode == BUFFER_IN && now - modeStart >= BUFFER_MS) setMode(FLUSH);
```

```
  if (mode == BUFFER_OUT && now - modeStart >= BUFFER_MS) setMode(STATE1);

  serviceKick(pump1, now);
  serviceKick(pump2, now);

  bool r = digitalRead(FLOAT_PIN) == FLOAT_ACTIVE;
  if (r != floatRaw) { floatRaw = r; floatChanged = now; }
  if (r != floatActive && now - floatChanged >= DEBOUNCE_MS) {
    floatActive = r;
    update();
    report();
  }

  if (pumpsRequested && (int32_t)(now - nextTick) >= 0) {
    uint32_t p = readPulses();
    uint32_t dp = p - lastP;
    lastP = p;
    nextTick += 1000;

    Serial.printf("%lu-%lu s [%s] pulses=%lu hz=%lu flow=%.3f L/min\n",
                  (unsigned long)sec, (unsigned long)(sec + 1), MODE_NAME[mode],
                  (unsigned long)dp, (unsigned long)dp, dp / CAL_K);
    sec++;

    if (mode == FLUSH && !p1NoFlowOff) {
      zeroSecs = (dp / CAL_K <= NOFLOW_LPM) ? zeroSecs + 1 : 0;
      if (zeroSecs > NOFLOW_S && !valveIsOpen) {
        p1NoFlowOff = true;
        Serial.printf("!!! flow <= %.2f L/min for more than %lu s with valve closed -> Pump-1 OFF\n"
            ,
                      NOFLOW_LPM, (unsigned long)NOFLOW_S);
        update();
        report();
      }
    }
  }
}
```